Project Sprint #3

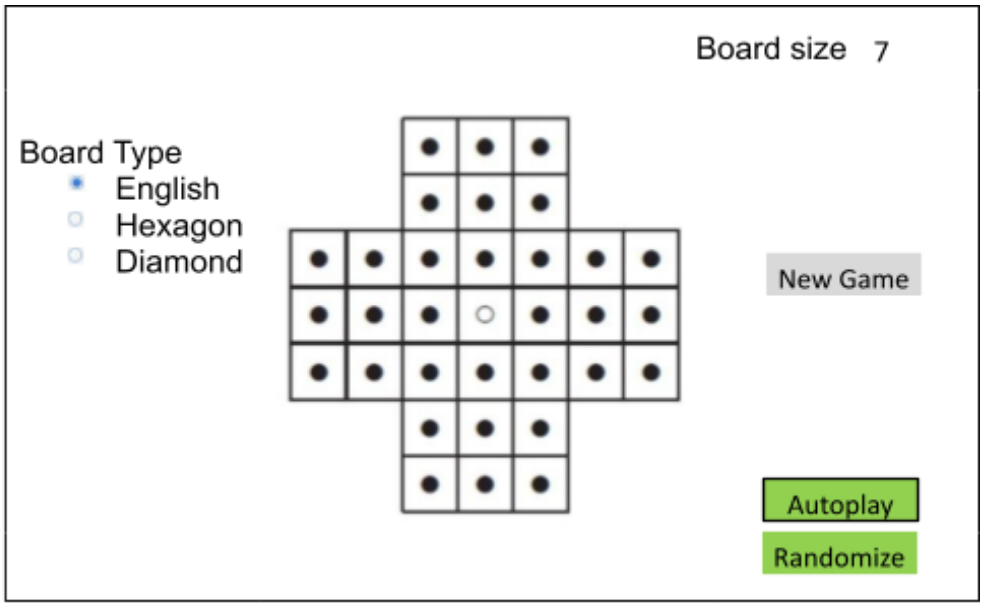
The Solitaire game is described in CS449HomeworkOverview.docx. You should read the description very carefully.

Your submission must include the GitHub link to your project and you must ensure that the instructor has the proper access to your project. You will receive no points otherwise.

GitHub link:

This sprint assumes that all the features that support a human player to play a manual or automated Solitaire game are already implemented. The minimum features include **choosing the game mode (manual or automated), choosing the board size, setting up a new game, making a move (in a manual or automated game), randomizing the state of the board** and **determining if a manual or automated game is over**.

The main task of this assignment is adding the feature of recording a game into a text file (or a lightweight database) and replaying from text file. The following is the complete GUI layout.



Deliverables: expand and improve your submission for sprint 2.

1. **Demonstration (9 points)**
   Submit a video of no more than five minutes, clearly demonstrating the following features.
   (a) A manual game until the game is over for English board
   (b) A manual game until the game is over for Hexagon board
   (c) An automated game until the game is over
   (d) A manual game with periodic randomization of states until the game is over
   (e) Some automated unit tests for the manual game mode
   (f) Some automated unit tests for the automated game mode

In the video, you must explain what is being demonstrated.
https://youtu.be/pPRsZcN22No

2. Summary of Source Code (1 points)

Source code file name	Production code or test code?	# lines of code
control.gd	Production	49
Grid.gd	Production	224
board_model.gd	Production	99
board_shape.gd	Production	63
	Total	432

You must submit all source code via github to get any credit for this assignment.

3. Production Code vs User stories/Acceptance Criteria (3 points)

Summarize how each of the user story/acceptance criteria is implemented in your production code (class name and method name etc.)

User Story ID	User Story Name
1	Choose a board size and type
2	Choose the game mode of a chosen board (manual/automated)
3	Start a new game of the chosen board size and game mode
4	Make a move in a manual game
5	A manual game is over
6	Make a move in an automated game
7	An automated game is over
8	Randomize the state of the board during a manual game

User Story ID	AC ID	Class Name(s)	Method Name(s)	Status (complete or not)	Notes (optional)
1) Choose board size and type	1.1	Control (via inheritance for all of Control)	_on_board_size_selection_item_selected()	Complete	Also starts new game
	1.2	Control	_on_board_type_selection_item_selected()	Complete	Also starts new game
	1.3	Control	_choose_board(type, size)	Complete	Also starts new game
2) Choose the game mode of a chosen board (manual/automated)	2.1	Control	_on_autoplay_toggled(toggled_on: bool)	Complete	Game starts off manually, autoplay can be toggled on/off midgame as well

3) Start a new game of the chosen board size and game mode	3.1	Control	_on_new_game_pressed ()	Complete	Automatically starts with preconfigured from the start
4) Make a move in a manual game	4.1	board_model	execute_move(from: Vector2i, to: Vector2i)	Complete	
	4.2	Control	show_moves(from: Vector2i)	Complete	
5) A manual game is over	5.1	board_model	is_game_over()	Incomplete	95% is complete Just need to make 1 more change
6) Make a move in an automated game	6.1	Control	_on_autoplay_tick()	Complete	
7) An automated game is over	7.1	Control	N/A	Complete	Uses the same checks as the manual
8) Randomize the state of the board during a manual game	8.1	Control	randomize_board()	Incomplete	Randomizes the empty center to somewhere else on the board but not the whole board itself.

4. Tests vs User stories/Acceptance Criteria (2 points)

Summarize how each of the user story/acceptance criteria is tested by your test code (class name and method name) or manually performed tests.

User Story ID	User Story Name
1	Choose a board size and type
2	Choose the game mode of a chosen board (manual/automated)
3	Start a new game of the chosen board size and game mode
4	Make a move in a manual game
5	A manual game is over
6	Make a move in an automated game
7	An automated game is over
8	Randomize the state of the board during a manual game

4.1 Automated tests directly corresponding to some acceptance criteria

User Story ID	Acceptance Criterion ID	Class Name (s) of the Test Code	Method Name(s) of the Test Code	Status (Pass/Fail/In Progress)
1) Choose board size and type	1.1	GutTest (via inheritance)	test_board_size()	Pass

4) Make a move in a manual game	4.1	GutTest board_model	test_can_move_valid_jump_square()	Pass
	4.2	GutTest board_model	test_cannot_move_if_midpoint_is_empty()	Pass
	4.3	GutTest board_model	test_execute_move_updates_state()	Pass
	4.4	GutTest board_model	test_invalid_execute_move_is_noop()	Pass
	4.5	GutTest board_model	test_find_any_move_returns_valid_pair()	Pass
5) A manual game is over	5.1	GutTest control	test_game_over_detection()	Pass
	5.2	GutTest control	test_stuck_board_is_game_over():	Pass
	5.3	GutTest control	test_winning_state_is_game_over()	Pass
7) An automated game is over	7.1	GutTest control	test_autoplay_terminates()	Pass

4.2 Manual tests directly corresponding to some acceptance criteria (if any)

User Story ID	**Acceptance Criterion ID**	**Test Case Input**	**Test Oracle (Expected Output)**	**Notes**
2) Choose the game mode of a chosen board (manual/automated)	2.1	Choose type "English" Choose size "5"	English board with 21 total cells with 4 corners missing should appear	Pass
3) Start a new game of the chosen board size and game mode	3.1	Click "New Game"	Reset with preconfigured settings	Pass
4) Make a move in a manual game	4.2	Click the 2nd most adjacent button on the right	Highlights the empty center (a valid move)	Pass
	4.3	Click on the adjacent button on the right	Do nothing	Pass
	4.3	Click on the empty button given the conditions of 4.2	Green button becomes Filled and Black, adjacent and origin button becomes empty and white	Pass

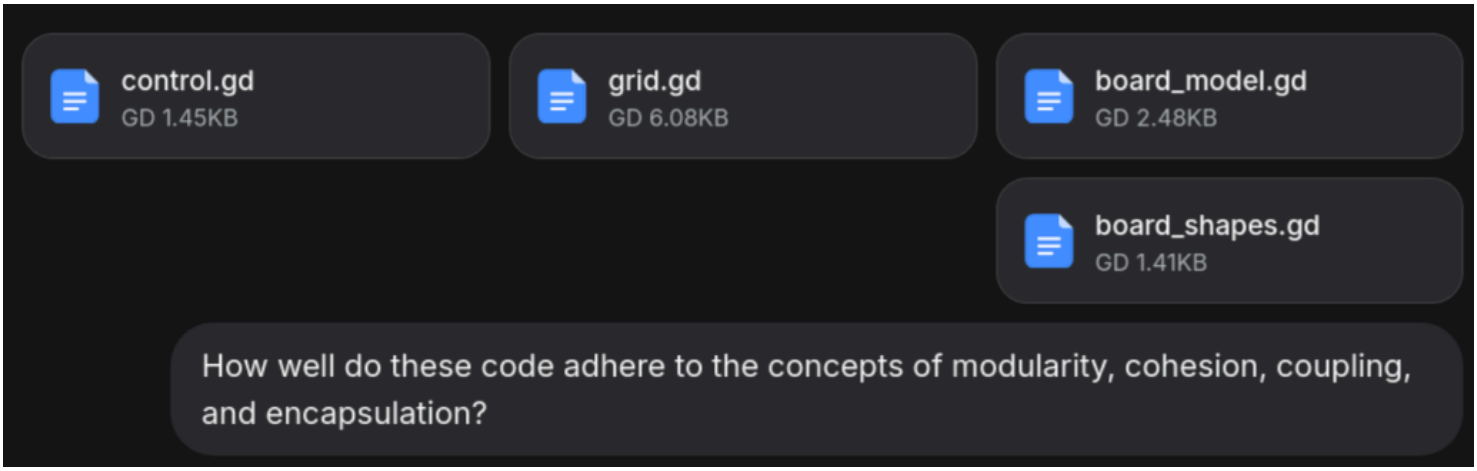
4.3 Other automated or manual tests not corresponding to the acceptance criteria (if any)

Number	**Test Input**	**Expected Result**	**Class Name of the Test Code**	**Method Name of the Test Code**

5. **(Part 1) Describe how the class hierarchy in your design deals with the common and different requirements of the Manual Game and the Automated Game.**
   At least 1/2 page, excluding screenshots/diagrams (12pt, single-spaced), is required. (3 Points)

(Part 2) Demonstrate how you use LLM (ChatGPT or other) to analyze how well your code adheres to the design principles discussed in class - modularity, cohesion, coupling, and encapsulation – using the definitions provided in class. Provide screenshots of your interactions with the LLM, showing the prompts used and the responses received.
Interpret the LLM's feedback, refine your prompts if needed to obtain more relevant responses, and discuss any changes made to your code based on the analysis. Explain how these refinements improve adherence to the design principles.
At least 1/2 page is required, excluding screenshots (12pt, single-spaced). (2 Points)

The code began with laying out the foundations of what the game looks like, thanks to the IDE that is GODOT, separation of the UI and Script was easy in all but one factor, With the more permanent features of the UI being easy to deal with. However and array of buttons I felt were very cumbersome to physically and manually set up as assets, so I used an array of Vectors to set up a board array of cells to initialize and iteration to set them up. The initialization step was charged with setting up the sizes and positions of each button, even creating a button group which had 1 important functionality: to limit the number of buttons that can be toggled to 1. For my solution to the autoplay there was simply no need to make a separate class since it could. class model and shapes were separated to reduce code and board initialization. Some of these design elements were automatically implemented thanks to Godot's IDE, there are base buttons, dropdown menus, and even the toggle button. With these clickable buttons binding them to the scripts was easy but I already found this script that deals with the buttons should be separate from the logic. At first the code was an array button implementation that needed repeating with the various shapes. But eventually that gets separated since that really never becomes retouched again. Though a flaw with GDScript is that there are no private class variables. So the calling functions outside the class can still access those class variables. I extended the board_shapes class to the board_model to make things easier, since in class the extended class inherits from the parent class and keeps those base functions as well and since I needed it to reuse code along with making things a bit more modular.


control.gd
GD 1.45KB
grid.gd
GD 6.08KB
board_model.gd
GD 2.48KB
board_shapes.gd
GD 1.41KB
How well do these code adhere to the concepts of modularity, cohesion, coupling, and encapsulation?

1. Modularity (How well are things separated into independent chunks?)

Score: 8/10

The Good:

- You have clear physical separation into three files: `board_shapes.gd` (geometry), `board_model.gd` (rules), and `grid.gd` (UI).
- `BoardShapes` has no dependencies whatsoever. It is a perfect standalone utility.
- `BoardModel` doesn't care about Buttons, Colors, or Nodes.

The Leak:

- `BoardModel` **extends** `BoardShapes`. This is a violation of modularity. A Model (which stores state) should not *be* a Shape utility. It should *use* it. By extending `BoardShapes`, `BoardModel` inherits all the static math functions, muddying its identity.
  Fix: Make `BoardModel` just `extends RefCounted`, and call `BoardShapes.english_board()` directly.

2. Cohesion (How closely related are the functions within a single module?)

Score: 9/10

The Good:

- `BoardShapes` is highly cohesive. Every function is a pure mathematical calculation.
- `BoardModel` is highly cohesive. Every function either changes the state of the dictionary (`set_empty`, `execute_move`) or queries it (`can_move`, `is_game_over`).
- `grid.gd` is cohesive in that it strictly handles UI (creating buttons, updating colors, handling clicks).

The Leak:

- `grid.gd` has a bit of "God Object" tendency. It handles input, button generation, color styling, *and* the autoplay timer. It's acceptable for a small project, but as the game grows, you might want to extract the autoplay logic into its own separate script or state machine.

3. Coupling (How much do modules depend on each other?)

Score: 7/10

The Good:

- `Grid` depends on `BoardModel` (one-way coupling). This is good. `BoardModel` has no idea `Grid` exists.
- `Grid` depends on `BoardShapes` for positioning math.

The Leak:

- **Data Leaking via `get_board_cells()`:** In `grid.gd`, your autoplay loop does `for from_cell in model.get_board_cells()`. You are pulling the internal array out of the model and looping over it in the UI. If `BoardModel` changes how it stores cells (e.g., switching to a flat array or a Set), `Grid` will break.
  Fix: Ask the model to do the work. Add a function to `BoardModel` called `get_available_moves()` that returns an array of `[from, to]` pairs. `Grid` just loops over that array.
- `Grid` **reaching out to the parent:** In `_on_autoplay_tick()`, you do `$"../GameOverPanel".show()`. This is tight coupling between `Grid` and the scene tree outside of it. If you move `Grid` to a different scene, this crashes.
  Fix: `Grid` should emit a custom signal `signal game_over_triggered` and let `Control.gd` (the parent) handle the UI.

4. Encapsulation (Hiding internal state and exposing a clean API)

Score: 6/10

The Good:

- `BoardModel` encapsulates the logic of *how* a move is executed. `Grid` just says `model.execute_move()` and trusts the model to handle the math.

The Leak:

- **Public Variables:** In `BoardModel`, you have `var board_cells: Array[Vector2i] = []` and `var cell_state: Dictionary = {}` completely public. In `grid.gd`, you directly access `model.board_cells.size()` and `model.board_cells[random_index]`. This means the UI has direct access to the Model's internal data structures. If the UI accidentally deletes a cell from `board_cells`, the dictionary `cell_state` will become desynced and the game will break.
  Fix: Make them private using underscores (`var _cell_state: Dictionary = {}`). Provide public getter functions (like `get_board_cells()`, which you already have, and use it!).

https://chat.deepseek.com/a/chat/s/11914411-2238-4a57-8463-d3813ca07f73

The concept of encapsulation was employed before I knew what it was since with my setup I had the board logic separate from the GUI since it just sounds like clutter to combine the two and admittedly the board setup and logic were still cluttered. The GUI script which deals with event handling does not have access to the variables that lie within the setup and logic handlers. While continuing the development of the program a lot of functions were outsourced to be encapsulated later on such as set_cell_color being used in various other functions such as refresh_highlights(), execute_moves(), restore_cell(). I've addressed the public variables before, but in class it seems that private class variables should never be able to be accessed at all. Some other functions alongside set_cell_color() such as set_filled() need this sort of modularity to decrease the complexity and help with problem solving oftentimes whenever it's needed by the AI and to prompt it for analysis. Most of the getter functions are fairly modular, the setters and getters for the class variables. Then there are the more complex setters and getters like for cell colors which are used by refresh_highlights() and update_cell_visual(), alongside other functions make_moves_. However it seems that using inheritance in this manner decreases its modularity. For cohesion, it's as fair as can be during the early part of the development of this project it was not as cohesive because planning classes out was not part of the plan and the grid.gd script did nearly everything short of handling the UI, from setting up the board, managing the moves, and automating the gameplay and thus scrolling up and down and using the find function was frequent. With some of the declared classes the board_shapes is able to consistently set up the shapes of the board whenever called and the board_model class deals with the board logic when making moves using the array of cells. The control.gd deals with both the GUI with user input and calls functions from the grid.gd whenever necessary without dealing with logic of the game and the grid.gd can call some functions to manipulate

the GUI but not with user input. The grid.gd expands more on the functionalities especially with the button array, randomization, and autoplay. Automated gameplay is simply just the automation of playing the game not anything else, therefore could be relegated to just a few functions that span 20 lines which is small enough to not matter, but that could change in the upcoming sprints. Now, thanks to the separation of the code into several smaller helper functions it helps with coupling. Before this the grid.gd script file did everything and now when it's split it becomes easier to manage coupling was poor, shape function calls using same setup calls without putting it in a function. Coupling now is much better though still has some ways to go.